Submodule 33.05: Handling Outliers (Visual Detection & Statistical Bounds)

Topic Overview & Theoretical Motivation

Outliers are extreme anomalies that deviate markedly from the general population manifold. In clustering algorithms:

- **K-Means Sensitivity**: Centroids are computed using arithmetic means (μ_k). A single extreme outlier pulls the cluster centroid toward itself, generating a spurious singleton cluster.
- **Hierarchical Clustering Sensitivity**: Linkage methods (such as single or average linkage) suffer from chaining or distorted dendrogram thresholds in the presence of extreme outliers.
- **PCA Sensitivity**: Principal component axes maximize variance; an extreme outlier dominates the first principal eigenvector, obscuring genuine cluster variance.

---

1. Visual Detection via Pairplots

To inspect multi-feature relationships and isolate outliers, we plot pairwise bivariate distributions across key metrics: `Income`, `Recency`, `Response`, `Age`, `Total_Spending`, and `Total_Children`.

Reference: smartcart.ipynb (Cell 22).

```
cols = ["Income", "Recency", "Response", "Age", "Total_Spending", "Total_Children"
sns.pairplot(df_cleaned[cols])
plt.show()
```

Visual Audit Observations:

1. **Age Outliers**: Multiple customer records display ages exceeding 120 years (birth years recorded as 1893, 1899, 1900). These are obvious typographical data entry errors.
2. **Income Outliers**: A distinct outlier customer exhibits an income of \$666,666, isolated far above the general population ceiling of \$160,000. This single point skews bivariate projections.

---

2. Statistical Filtering & Boundary Selection

We impose two domain-consistent statistical bounds:

1. $\text{Age} < 90$ years: Excludes historic birth-year typographical errors.
2. $\text{Income} < \$600,000$: Filters out the anomalous \$666,666 outlier.

Best Engineering Practice (Preserving State):

Always preserve intermediate dataframe states (`df_cleaned`, `df_encoded`, `X_scaled`, `X_pca`) rather than in-place overwriting (`inplace=True`). This enables step-by-step verification, downstream debugging, and pipeline reversibility.

---

3. Python Implementation

Reference: smartcart.ipynb (Cell 23).

```
print("Data size with outliers:", len(df_cleaned)) # 2240

# Filter outliers based on demographic and financial bounds
df_cleaned = df_cleaned[(df_cleaned["Age"] < 90)]
df_cleaned = df_cleaned[(df_cleaned["Income"] < 600_000)]

print("Data size without outliers:", len(df_cleaned)) # 2236
```

Impact:

- **Rows Filtered**: Exactly 4 rows dropped (2,240 $\rightarrow$ 2,236).
- **Integrity**: 99.82% of customer records preserved while eliminating centroid-distorting extremes.